

Machine Learning Techniques for EEG

Dr. Mohamed H. Zaky, Assistant Professor

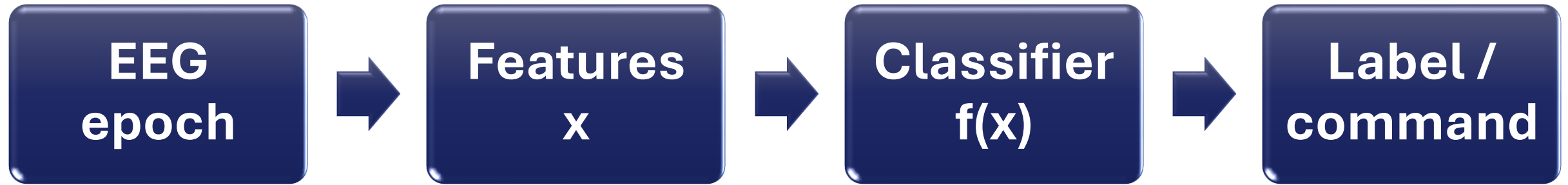
Co-founder - WeBios Lab

Electronics and Communications Engineering Department
Arab Academy for Science, Technology and Maritime Transport



HERBioLab

From features to decisions



Supervised learning

- Each training trial carries a known label.
- Target vs non-target, left vs right hand, rest vs task.
- Fit parameters on training data, then predict on new trials.

The real goal

- Perform on unseen trials, sessions and people.
- High training accuracy with low test accuracy = overfitting.
- The **evaluation protocol** matters as much as the model.

What a classifier actually learns?

A classifier **draws decision boundaries** in feature space; training **moves those boundaries** to best separate the classes.

Inputs → outputs

- **Input:** A feature vector x (or a raw epoch).
- **Output:** A class label y .
- **Learns:** A boundary, or the probability $P(y|x)$.

Train → test

- **Training:** Learns patterns from the primary labeled dataset.
- **Validation:** Fine-tune model settings during training.
- **Testing:** The completely frozen model faces unseen data.

Why EEG is problematic for machine learning?

Four pressures shape almost every design choice that follows.

Low SNR and artifacts

- Brain signal is buried in background activity.
- Eye blinks, muscle, and line noise dwarf the effect of interest.

Non-stationarity

- Signal statistics drift within and across sessions and days.
- A model trained today can degrade tomorrow.

High-dim and few trials

- Many channels $\times$ time / features, but only tens–hundreds of trials.
- Curse of dimensionality $\rightarrow$ easy overfitting.

Subject variability

- Anatomy and mental strategy differ between people.
- A model tuned to one subject transfers poorly to others.

Two ways to decode EEG

The same trial can be classified from **hand-made features** or **straight from the signal** – two philosophies with different trade-offs.

On typical BCI datasets (tens–hundreds of trials) the classic pipeline is often the stronger, safer baseline. Start there.

Classic machine learning pipeline

Recipe: Hand-crafted features → LDA / SVM.

Strengths: Works with little data, fast, and interpretable.

Cost: Needs domain knowledge to design good features.

Typical use: Single-subject, online BCI.

End-to-end deep learning

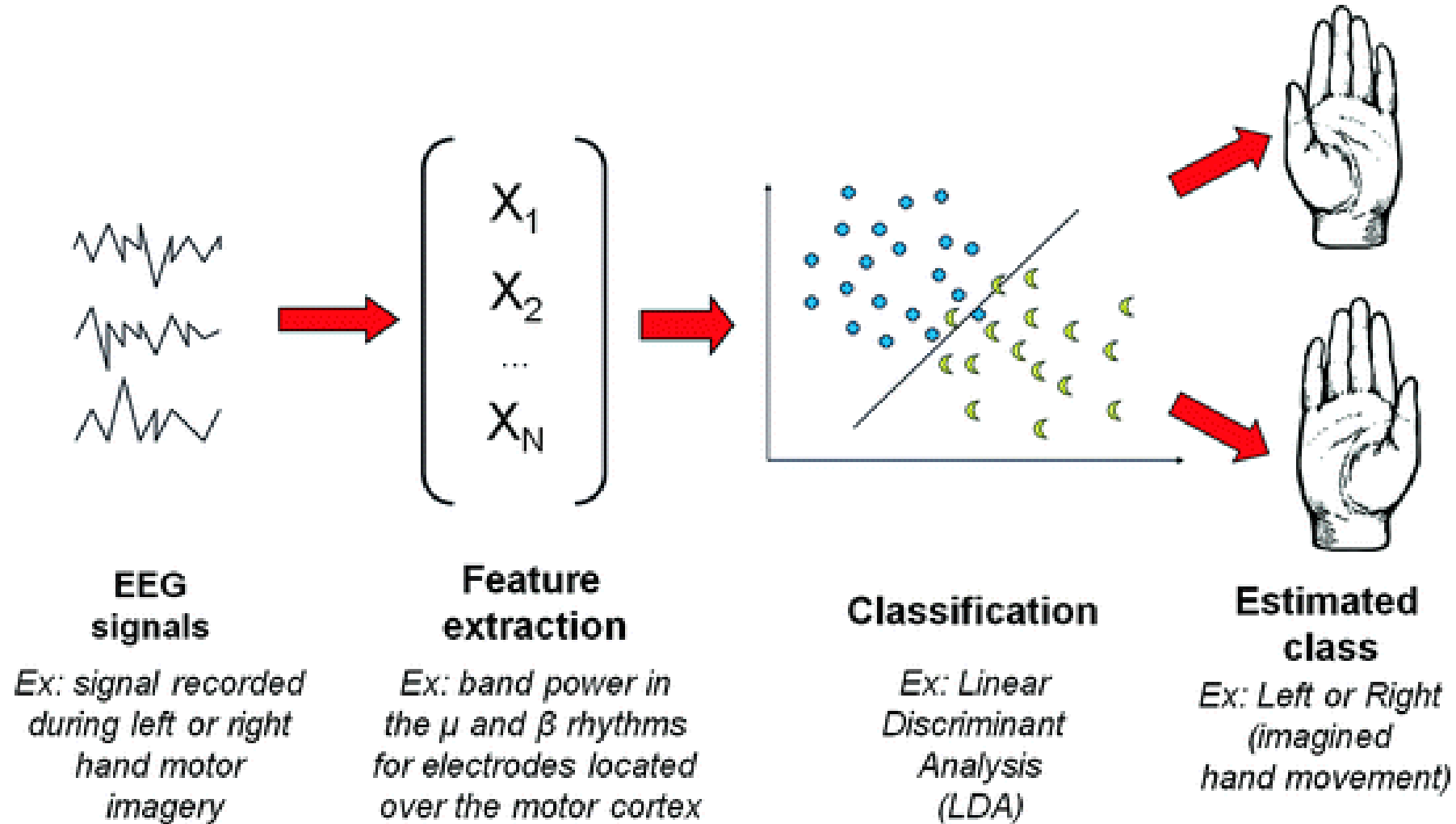
Recipe: Raw / lightly-processed EEG → network learns features + classifier.

Strengths: No manual features, scales with data.

Cost: Needs more data and compute, harder to interpret.

Typical use: Large or pooled multi-subject datasets.

Linear classification models



Linear discriminant analysis (LDA)

The default first classifier in BCI: a linear boundary built from class means and a shared covariance.

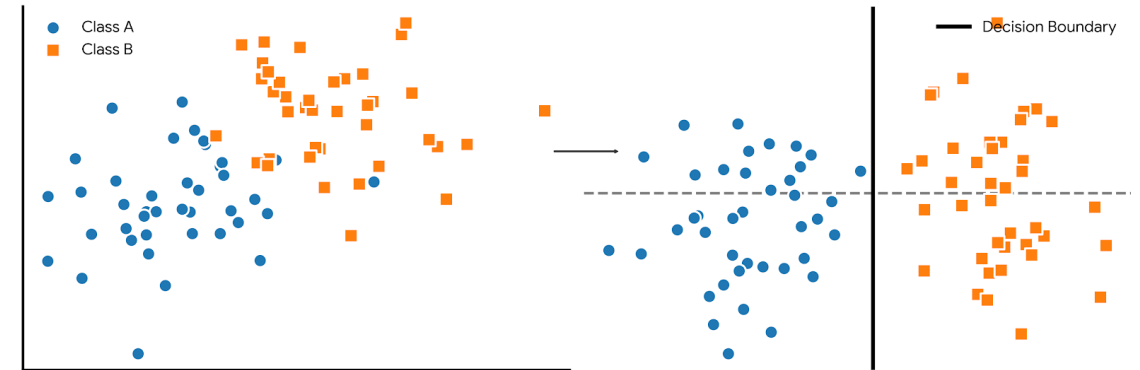
LDA is the **baseline** every new BCI method should be measured against.

Why use it

- Almost **no hyperparameters** to set.
- Trains in milliseconds; ideal for **online use**.
- Stable with **few trials** and hard to overfit.

Where it struggles

- Fails when the true boundary is **non-linear**.
- Sensitive to a poor estimate (**many features**).



Support vector machines (SVM)

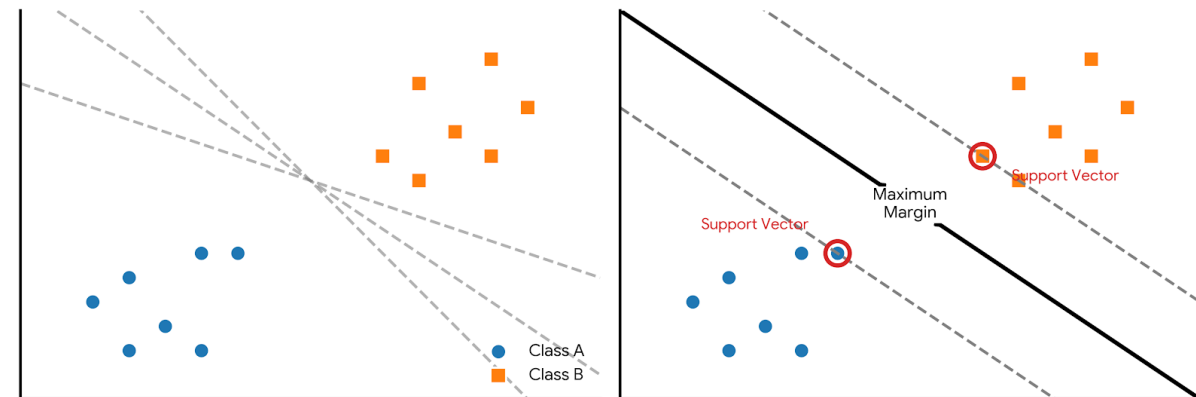
SVMs pick the boundary with the widest possible margin between classes and can bend it with kernels.

Why use it

- SVM uses kernel functions to **map out non-linear patterns**.
- SVM remains **highly stable and effective** even before feature matrices' dimensions reduction.
- It mathematically calculates the **widest possible gap** between classes which helps **filter out random noise**.

Where it struggles

- Complex kernel math **requires heavier computation** which makes it less ideal for real-time streaming compared to LDA.
- If you do not **standardize your data first**, larger brainwave signals will completely break the model's accuracy.



LDA or SVM? Choosing a linear decoder

Both are excellent baselines; the data – **not habit** – should pick the winner.

Prefer LDA when

Trials are very few.

You need speed and simplicity (online BCI).

Features are direct and simple.

You want an almost parameter-free baseline.

Prefer SVM when

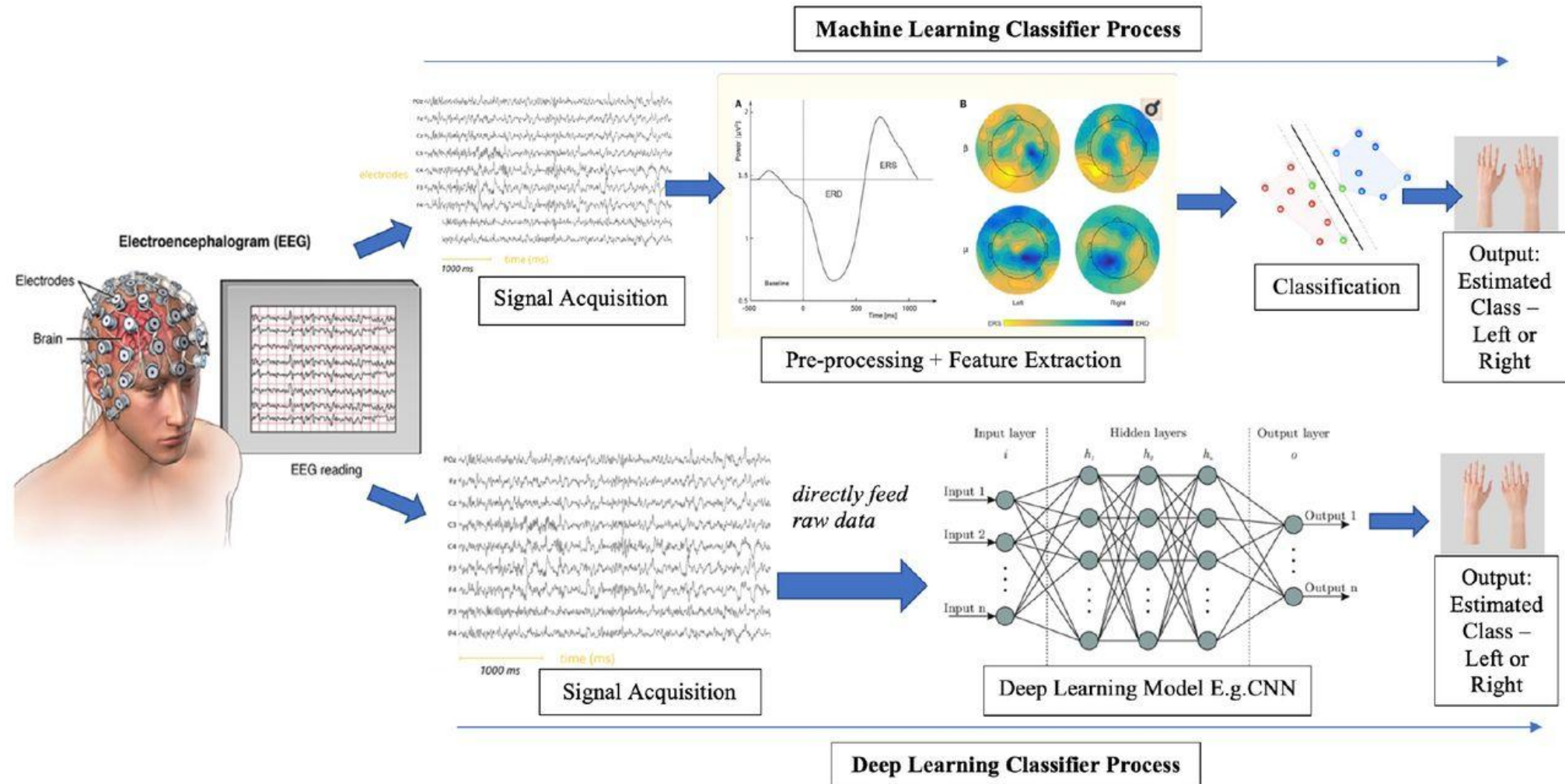
More trials are available to tune on.

The boundary may be non-linear.

Features are high-dimensional and sparse.

Margin robustness to outliers matters.

Non-linear classification models



Convolutional neural networks (CNN) for EEG

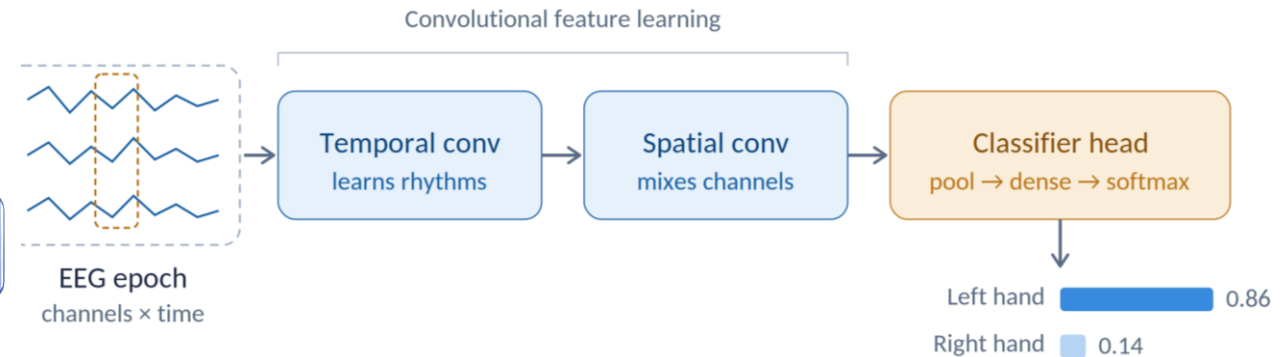
CNNs slide **small learnable filters** over the signal, discovering temporal rhythms and spatial patterns automatically.

Why use it

- The network **automatically learns** to extract its own optimal features directly from raw data.
- It treats the layout of EEG sensors like a geographical map (spectrogram). This allows it **to capture how electrical activity physically flows across different regions of the scalp**.

Where it struggles

- Because gathering thousands of clean, artifact-free EEG trials per subject is incredibly difficult, **CNNs often overfit on small BCI datasets**.
- This **lack of biological transparency** makes verifying hardware integrity and debugging errors highly frustrating.



Classical machine learning or deep learning?

Pick the family that matches your data budget and deployment needs, **not the most fashionable one**.

Always run a **shrinkage-LDA/SVM baseline** first; adopt deep learning **only if** it clearly and repeatably beats it.

Reach for classical machine learning when

Labelled data is small or medium.

You need interpretability.

The decoder must run fast, online.

Compute is limited – most single-subject BCI.

Reach for deep learning when

You have large or pooled datasets.

You want to learn from raw multichannel signals.

You can transfer from a pretrained model.

Accuracy outweighs interpretability.

Transfer learning strategy: Pretrain and adapt

The baseline (Pretrain)

Instead of starting from zero, the system first **trains a base machine learning model using a massive, generalized** dataset collected from **dozens** of different subjects.

This teaches the algorithm the **universal, baseline patterns of human brainwaves**.

The customization (Adapt)

Because every person's **brainwaves are slightly different** (like a fingerprint), the generalized model **won't perfectly** fit a new user.

The system **records a tiny amount of the new user's specific EEG data** and uses it to **fine-tune** or "**adapt**" the pretrained model to their unique neural signature.

Why transfer learning is essential for BCIs

Solves data scarcity

It allows the classification system to achieve high accuracy for a specific subject **even if you only have a few minutes of their recorded data.**

Fixes daily drift

A user's **brainwave patterns naturally shift from day to day** due to fatigue, stress, or slight changes in how the headset is worn.

Transfer learning allows the system to **rapidly adapt to these daily changes** without requiring a full system recalibration.

Pretrained tools and foundation models

EEGNet is an architecture you train yourself; foundation models ship pretrained weights you fine-tune.

These are research models with open weights are powerful but still evolving; not always better than a strong baseline on small data.

Model	Year	Core idea	Pretraining
EEGNet	2018	Compact CNN architecture (not pretrained)	You train it
BENDR	2021	First self-supervised EEG (contrastive)	Large clinical EEG
BIOT	2023	Tokenizes biosignals, cross-dataset	Mixed biosignals
LaBraM	2024	Neural tokenizer + masked modeling	~2,500 hours
EEGPT	2024	Dual self-supervised transformer	Large EEG corpus
CBraMod	2025	Criss-cross spatial/temporal attention	~9,000 hours
REVE	2025	Adapts to any montage; largest corpus	~60,000 hours

Data splitting: Real accuracy vs. illusion

How you divide your dataset determines whether your model's performance is **scientifically valid** or just a **mathematical mirage**.

Choosing the right strategy

Within-subject: Training and testing **exclusively** on a single person's data to build a highly customized model.

Cross-subject: Training on a large group and testing on a completely unseen person (**Leave-One-Out**) to prove true real-world generalization.

Small datasets: Use **K-Fold Cross-Validation** to repeatedly slice, train, and test the data, ensuring you evaluate the **average performance** and the spread.

Preventing data leakage

The golden rule: Every mathematical step that learns from the data must be calculated on the **training set alone**.

Protect the test set: Your testing data is sacred. **Never** use it to tune model parameters.

Maintain pure boundaries: If you are doing **cross-subject validation**, ensure an individual person's trials **never bleed** across the train/test divide.

Hyperparameter tuning: Configuring the model

The core concept

- While **parameters** are the patterns the **algorithm learns from the data**, **hyperparameters** are the **manual control** dials you must set **before training** begins.

What to tune in each model

- **LDA**: Adjust the **shrinkage level** to manage how the model handles variance.
- **SVM**: Select the **kernel type** and **tune the C and gamma** values to control the strictness of the decision boundary.
- **Neural Networks**: Configure the **learning rate, network architecture (depth and width), dropout rate** to prevent overfitting, batch size, and total training epochs.

Hyperparameter tuning: Configuring the model

How to search for the best settings

- **Grid search**: Exhaustively tests every single possible combination. This is thorough but only practical for very small parameter spaces.
- **Random search**: Randomly samples different combinations. It is significantly faster and more efficient when you have many parameters to tune.
- **Bayesian optimization**: A "smart" search that uses the results of past tests to logically guide where it should guess next.
- **Early stopping**: A technique specifically for Neural Networks that automatically halts training as soon as the model stops improving.

Tuning safely

- **Use validation sets**: Only test your hyperparameter adjustments on a dedicated validation fold or through nested cross-validation.
- **Protect the test set**: Your final test data must remain completely untouched during the tuning phase.

Data augmentation: Expanding your dataset

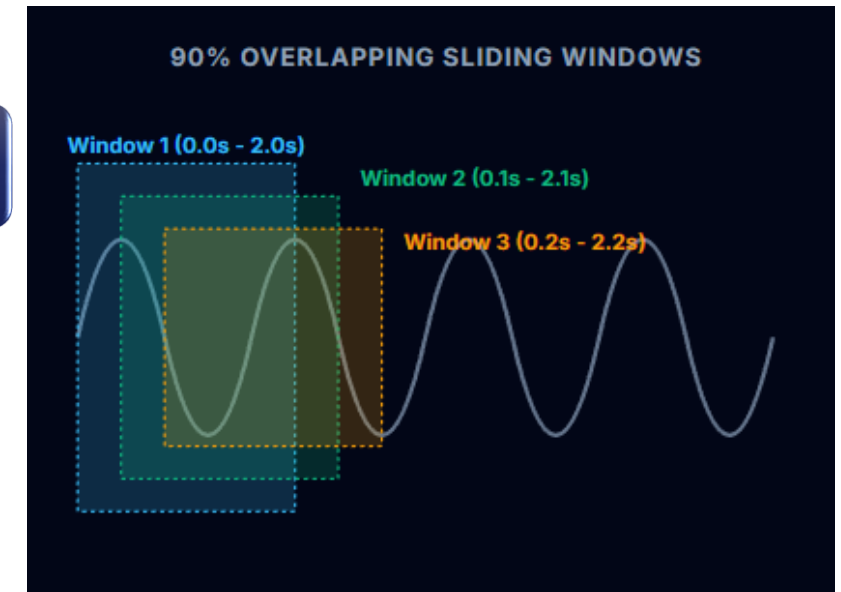
Augmentation combats data scarcity by **mathematically varying your existing trials to create new training examples**. This improves the model's robustness while preserving the original cognitive label.

Techniques for augmentation

- **Noise and scaling:** Injecting **minor Gaussian noise** or slightly **adjusting the amplitude** (jittering) of the existing brainwaves.

Limitations and cautions

- **The train-only rule:** You must strictly **apply augmentation only to your training set**. **Never** artificially alter your **testing data**.
- **Maintain biological reality:** **Over-distorting the signal creates physiologically impossible brainwaves** that will break the model.
- **No substitute for reality:** While a highly useful mathematical approximation, **artificial augmentation can never fully replace the value of recording genuine human EEG data**.



Preparing a dataset for a classifier

Most BCI mistakes are made before training starts – in how the data is shaped, labelled and scaled.

Epoch and label

Cut trials around events and attach the correct class label to each one.

Shape the tensor

Feature vector per trial (classic machine learning), or trials $\times$ channels $\times$ time (deep learning).

Standardize

Z-score/normalize – fit the scaler on **training data only**, then apply to val/test.

Balance classes

Address **imbalance** with resampling, class weights or augmentation.

Keep metadata

Track subject, session and time so splits can respect them.

Evaluation: Choosing the right metrics

The core concept

- Relying on **raw accuracy** alone is dangerous. It creates a false sense of success, easily hiding a failing model if your dataset is imbalanced or if the probability of random guessing is high.

Going beyond raw accuracy

- For imbalanced data: Use **Balanced Accuracy, F1-Score, and ROC-AUC** to mathematically ensure the model isn't just **lazily predicting the most common cognitive state**.
- For randomness: Use **Cohen's Kappa** to measure true learning by factoring out the percentage of predictions the **model got right purely by chance**.
- The ultimate check: Always review the **Confusion Matrix** to understand exactly **where and how your model is making mistakes**.

Evaluation: Choosing the right metrics – BCI-related

BCI-specific evaluation

- **Information transfer rate (ITR)**: The gold standard for BCIs. It measures real-world utility by **combining both speed and accuracy to calculate the actual bandwidth** of command data a user can output.
- **The reality gap**: Always note that a **model's offline performance will almost certainly differ from its performance in online streaming environment**.

Best practices for reporting

- **Show the full picture**: Never report a single, cherry-picked "headline" number from a lucky data split.
- **Prove consistency**: Report the mean performance and the variability (the standard deviation) across all subjects to prove your model's true real-world robustness.

A practical decision guide

A starting point, not a rule book – always make the choice based on your own data.

Your situation	Sensible first approach
Few trials, needs to run fast/online	Shrinkage-LDA (or Riemannian + LDA)
More data, boundary may be non-linear	SVM with an RBF kernel, or a small CNN
Raw multichannel signal, moderate data	EEGNet – the proven deep baseline
Large, pooled or cross-subject data	Fine-tune a foundation model
Very scarce labels	Augmentation + pretrained features
<i>Every project, without exception</i>	<i>Honest split · baseline first · tune on validation</i>

Muse 2 offline pipeline

Building a classification model

Summary of utilized libraries

sklearn

The core machine learning engine. It is used to split the dataset, scale the features, train the classifiers (LDA and SVM), validate robustness (Stratified K-Fold), and calculate performance metrics (Accuracy, Cohen's Kappa, ROC-AUC, etc.).

joblib

Used for model serialization. It **packages the trained model, the standard scaler, and the feature names into a single lightweight .joblib file** for immediate deployment in the live streaming environment.

pandas

Handles the ingestion and structural management of the extracted feature CSV.

numpy

Handles the complex array mathematics, specifically required to calculate the ITR using logarithmic functions.

tkinter

Provides the GUI to securely prompt the operator for the subject ID without relying on console inputs.

The script's pipeline

Loads the CSV and isolates the targeted cognitive features (X) from their binary class labels (y).



Divides the data into 80% training and 20% holdout testing sets, preserving the class ratio, and normalizes the data distribution using standard scaling.



Independently trains an LDA and an SVM, using 5-Fold Cross-Validation to assess stability.



Tests the final trained models on the unseen 20% holdout set, computing rigorous BCI metrics (including ITR and Cohen's Kappa).



Exports both the trained models and the fitted scaler as deployable .joblib files directly into the subject's directory.

Data ingestion and preprocessing

Subject-specific targeting

- The pipeline utilizes a GUI to load only the highly optimized features extracted and mathematically pruned (via ANOVA) in the previous pipeline stage.

Splitting

- The dataset is divided into an **80%** training block and a **20%** holdout testing block. This ensures the exact ratio of resting to active states is maintained across both sets to prevent bias.

Standard scaling

- Features like Beta and Alpha power naturally exist on **different numerical scales**. The *StandardScaler* **standardizes** these features, so they share a mean of zero and unit variance. This prevents larger numerical values from dominating the algorithm.

LDA vs. SVM (Which is better for our pipeline?)

The default choice (LDA)

- If the comparative report shows that **LDA matches or even slightly trails SVM**, keep the **LDA**.

The exception (SVM)

- You should only select the SVM if **its holdout accuracy vastly outperforms the LDA by a significant margin**.

The final verdict

- **For dry-sensor BCI applications** utilizing focused feature pruning (like top-2 ANOVA selection), **the simplicity and speed of LDA generally make it the superior**, more robust choice for production deployment.

LDA hyperparameters

```
LinearDiscriminantAnalysis(solver='lsqr',  
shrinkage='auto')
```

- **shrinkage='auto' (Automatic regularization):** This parameter automatically calculates the optimal penalty to **shrink extreme values, preventing the linear model from overfitting** to random electrical noise within the epoch.
- **solver='lsqr' (Least squares solver):** The default 'svd' solver does not mathematically support shrinkage. Switching to 'lsqr' strictly enables the regularization feature required to stabilize the model.

Kernel SVM hyperparameters

```
SVC(kernel='rbf', C=1.0, gamma='scale', probability=True, random_state=42)
```

- **kernel='rbf' (Radial basis function):** Maps the data into a higher-dimensional space to detect complex, **non-linear** relationships that the LDA might miss.
- **C=1.0 (Regularization penalty):** Sets a balanced penalty for misclassifications, **preventing the model from drawing overly convoluted boundaries** just to force a 100% training score.
- **gamma='scale' (Kernel coefficient):** Automatically adapts the influence of single training examples based on the number of features and the variance of the scaled data.
- **probability=True (Probability estimates):** SVM do not output probabilities by default. This is forced to True so the script can **mathematically calculate the ROC-AUC evaluation score**.
- **random_state=42 (Seed lock):** Locks the **random number generator** used during probability CV to **guarantee exact reproducibility** for the participants across different runs.

System robustness (Cross-Validation)

What it means

The script uses **5-Fold Cross-Validation**, meaning it slices the **training data** into five chunks, trains on four, and tests on one, repeating this five times.

The report shows the **average accuracy** across these five tests and how much the accuracy fluctuated (**the spread**).

How to interpret

High mean/low spread: The **ideal** scenario. The model is stable, reliable, and learns consistent patterns regardless of how the data is sliced.

High spread: If the **spread is large** (e.g., $\pm 15\%$), the model is **highly unstable**. It is likely memorizing specific noise artifacts in certain data chunks (overfitting) rather than learning true brainwave patterns.

Primary holdout metrics (Accuracy and F1)

What it means

Raw accuracy: The total **percentage of correct predictions** based on the testing set.

Balanced accuracy: The mathematical **average of the true positive rate and true negative rate**, adjusting for **uneven datasets**.

F1-score: The harmonic mean of precision (**quality of predictions**) and recall (**ability to find all targets**).

How to interpret

If your raw accuracy is exceptionally high (e.g., 90%) but your balanced accuracy or F1-score drops significantly (e.g., 60%), your model is **cheating**.

It indicates a **class imbalance** where the model is simply guessing the **most common** state (like "Resting") constantly and failing to actually detect the cognitive load.

Advanced BCI metrics (Kappa and ITR)

What it means

Cohen's Kappa: A statistical metric that accounts for the **possibility of the model getting answers right purely by random chance**.

ITR: The gold standard metric for BCIs. It mathematically **combines the model's accuracy, the number of target classes, and the duration of the trial into a single measurement of bandwidth speed**.

How to interpret

A Kappa score **below 0.4** indicates **poor reliability**, while a score above **0.6–0.8** indicates **substantial**, trustworthy predictive power.

ITR tells you the practical utility of the model. If you are building a live system (like a neural keyboard or wheelchair control), **ITR tells you exactly how much actual command data the subject can push through the system every sixty seconds**.

Confusion matrix and ROC-AUC score

What it means

The confusion matrix explicitly **counts every** True Positive, True Negative, False Positive, and False Negative.

The ROC-AUC (Receiver Operating Characteristic - Area Under Curve) **evaluates the model's ability to distinguish between the two classes across various probability thresholds**.

How to interpret

The confusion matrix: Look specifically at **how the model fails**. Is it incorrectly flagging "Rest" as "Focus", or is it missing the "Focus" state entirely? **This tells you if your biological baseline is drifting or if the cognitive task was too difficult.**

ROC-AUC: A score of **0.50** means the model is flipping a coin. A score of **1.0** means **perfect mathematical separability** between classes.

Automated deployment and reporting

Model packaging

- Packages the winning model and its settings into a single deployment file (**.joblib**).

Integrated scaling

- A scaler simply shrinks messy raw data into a uniform size (e.g., between 0 and 1).
- It must be saved with the model so that future, live brainwaves are resized using the exact same math before the model tries to read them.

Diagnostic export

- Automatically generates text reports so you can instantly compare algorithm accuracy (like LDA vs. SVM) and review confusion matrices.

Technical advice

In practical BCI applications, a model with **85% accuracy** that can make a decision every **1.5 seconds** is **vastly superior** to a model with **95% accuracy** that requires **4 seconds** of data accumulation.

Evaluate your models based on the **ITR (bits/min)** rather than **raw accuracy** alone.

Head-to-head comparison matrix example

=====

HEAD-TO-HEAD BCI CLASSIFIER COMPARISON (SUBJECT 001 - MATH)

=====

Target Directory: D:\MuseData\ID_001\offline

METRIC	REGULARIZED LDA	RBF KERNEL SVM

CV Mean Accuracy	82.50% (± 4.2%)	84.10% (± 3.8%)
Holdout Accuracy	85.00%	86.50%
Balanced Accuracy	84.80%	86.20%
ROC-AUC Score	0.8950	0.9120
Cohen's Kappa	0.7000	0.7300
F1-Score	0.8570	0.8710
ITR (bits/min)	12.50	14.20

CONCLUSION DIAGNOSTIC:

- If SVM Holdout significantly outperforms LDA, your EEG features possess non-linear relationships.
- If LDA matches or beats SVM, keep LDA. It is faster, less prone to overfitting, and better suited for real-time BCI.

=====

How to interpret the head-to-head comparison matrix

CV mean accuracy (82.50% ± 4.2%): The 5-fold cross-validation average. This means when the **training data was chopped into 5 pieces and rotated**, the LDA model correctly identified the brain state 82.5% of the time, with a 4.2% variance between the easiest and hardest data slices.

Holdout accuracy (85.00%): How well the model performed on the **completely unseen 20% testing split**. This is the most **realistic** expectation for how the model will perform live.

Balanced accuracy (84.80%): Averages the accuracy of each class individually. This ensures the model is learning both states and not just cheating by guessing the most common class in an unbalanced dataset.

Cohen's Kappa (0.7000): This metric accounts for **random guessing**. A score of **0.0 is random chance, and 1.0 is perfect**. A 0.7000 shows substantial agreement between the EEG data and the model's predictions.

ROC-AUC score (0.8950): Measures the **model's absolute ability to distinguish between classes**. A score approaching **1.0** means the model has a very clean boundary between states.

ITR (12.50 bits/min): ITR calculates how much **effective communication bandwidth** the user has, factoring in both accuracy and the trial duration.

Understanding the F1-Score

Precision: Trusting the trigger

- The **metric**: $\text{True Positives} / (\text{True Positives} + \text{False Positives})$
- **What it means**: When the BCI predicts an "Active" state, **how often is it actually right?** High precision minimizes false positives, preventing accidental commands when the user is just daydreaming.

Recall (Sensitivity): Catching the command

- The **metric**: $\text{True Positives} / (\text{True Positives} + \text{False Negatives})$
- **What it means**: Out of all genuine "Active" states, **how many did the model successfully detect?** **High recall minimizes false negatives**, ensuring the system registers faint but intentional commands without dropping them.

The trade-off and F1-Score (0.8570)

- There is an inherent tug-of-war: tuning for **perfect recall triggers on background noise**, while tuning for **perfect precision misses valid commands**.
- The F1-Score **mathematically balances both halves**. A score of 0.8570 confirms the model successfully catches genuine brain states without being overly sensitive and triggering false alarms.

How to interpret the confusion matrix

True Negatives (Top Left - 42): Out of **50** actual relaxing epochs, the model correctly predicted **42**.

False Negatives (Bottom Left - 7): The user was actively doing the task (focusing), but the model mistakenly thought they were relaxing **7** times.

True Positives (Bottom Right - 43): Out of **50** active focus epochs, the model successfully detected **43**.

False Positives (Top Right - 8): The user was actually relaxing, but the model falsely thought they were focusing **8** times.

CONFUSION MATRIX

	Pred Relax	Pred Focus
Actual Relax :	42	8
Actual Focus :	7	43

*Thank
You!*



HERBioLab
